

污染场地土壤生态-生产功能重构监管系统 运维交接手册

文档版本：v0.1

编写日期：2026-06-13

编写人：辛特助（系统总架构师 / 全栈负责人 / ML 算法负责人）

密级：内部

适用范围：开发环境 / 测试环境 / 生产部署

第一章 系统概述

1.1 系统定位

"污染场地土壤生态-生产功能重构监管系统"是一个面向政府监管与企业治理的污染场地全流程监管平台。系统核心能力为：数据可信、诊断可解释、评价可追溯、方案有依据、流程可监管。核心交付物为单场地全流程监管追溯报告（PDF）。

1.2 业务域与功能模块

业务域	功能模块	关键能力
数据管理	场地数据录入/导入 (Excel/CSV)	字段映射、异构表头兼容、 数据校验
数据管理	检测数据长表存储	measurements 表，一污染物一行
数据管理	场地列表/详情/查询/筛选	分页、排序、多维度筛选
数据管理	数据导出与归档	导出包含来源溯源信息
数据管理	图表与地图可视化	ECharts + Leaflet/PostGIS
决策管理	关键障碍因子识别 (RF+SHAP)	Top-N 因子、全局/局部 SHAP 解释
决策管理	功能重构可行性评价	生产/生态双方向，改进模糊综合评价法
决策管理	SSUI 可持续利用评价	安全+经济维度、时间权重、管理调节
决策管理	重构方案推荐	规则引擎 + 技术库匹配
全流程追溯	五阶段追溯记录	调查评估/方案审批/施工监理/效果评估/后期管护
全流程追溯	一键 PDF 追溯报告生成	15 项内容完整报告，版本快照可复现
系统管理	RBAC 四角色权限	管理员/企业用户/第三方机构/监管人员
系统管理	操作日志与审计	所有写操作记录、IP/UA 溯源
系统管理	参数/阈值/模型/技术库配置	全部入库或入配置，不硬编码

1.3 技术栈总览

层级	技术选型	版本要求	用途
前端	React + TypeScript	React 18+, TS 5+	SPA 单页应用
前端	Ant Design	5.x	UI 组件库
前端	ECharts	5.x	数据可视化图表
前端	Leaflet	1.x	GIS 地图交互
后端	Python + FastAPI	Python 3.10+, FastAPI 0.115+	REST API 服务
后端	SQLAlchemy + Alembic	2.0.36+, 1.14+	ORM + 数据库迁移
后端	Pydantic	2.9+	数据校验与序列化
数据库	PostgreSQL	15+	主数据库 (可扩展 PostGIS)
数据库	Redis	7.x	缓存与会话
文件存储	本地文件系统	-	MVP 阶段文件存储 (后续可切 MinIO)
ML	scikit-learn	1.5.2+	随机森林模型
ML	SHAP	0.47+	模型可解释性
报告	Jinja2 + WeasyPrint	3.1+, 63+	HTML 模板渲染 → PDF 生成
部署	Docker + Docker Compose	Docker 24+	容器化编排

第二章 部署指南

2.1 环境要求

组件	最低版本	检查命令	备注
Docker	24.0+	docker --version	含 Docker Compose V2
Docker Compose	2.x	docker compose version	V2 插件，非 docker-compose (V1 已弃用)
磁盘空间	≥10 GB 可用	df -h	PostgreSQL 数据卷 + 文件存储
内存	≥4 GB	free -m	含 PostgreSQL + Redis + Backend
macOS	13 Ventura+	sw_vers	Apple Silicon (ARM64) 或 Intel (x86_64)
Linux	Ubuntu 22.04 LTS+ / CentOS Stream 9+	uname -a	需要 systemd 或等效 init

2.2 项目目录结构（部署相关）

项目根目录：/Users/lensis/Claude/Projects/SRS/

关键部署路径：

deploy/docker-compose.yml — Docker 编排文件 (PostgreSQL + Redis + Backend)

backend/Dockerfile — 后端镜像构建文件 (Python 3.11-slim)

backend/requirements.txt — Python 依赖清单

backend/alembic.ini — 数据库迁移配置

backend/alembic/ — 迁移脚本目录

backend/app/core/config.py — 环境变量配置 (Pydantic Settings)

frontend/dist/ — 前端构建产物 (桌面打包模式自动挂载)

data/ — 数据目录 (raw/processed/model_ready/knowledge_base)

2.3 首次部署步骤

2.3.1 克隆代码仓库

```
cd <部署目录>

git clone <仓库地址> SRS
cd SRS
```

2.3.2 创建环境变量文件

在 deoploy/ 目录下创建 .env 文件，内容模板如下：

```
# — 数据库 —
POSTGRES_USER=srs
POSTGRES_PASSWORD=<生成强随机密码，至少 16 位>
POSTGRES_DB=srs

# — Redis — ( 使用默认端口，无需额外配置 )

# — 后端 —
SECRET_KEY=<生成强随机密钥，至少 32 位，用于 JWT 签名>

# 可选覆盖：
# DATABASE_URL=postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}
# @db:5432/${POSTGRES_DB}
# REDIS_URL=redis://redis:6379/0
# FILE_STORAGE_DIR=/app/storage
```

生成安全密钥的命令：

```
# 生成 32 字节随机密钥 (Base64 编码)
openssl rand -base64 32
```

注意：.env 文件禁止提交到 Git，已在 .gitignore 中排除。

2.3.3 构建前端（如需要桌面打包模式）

如 frontend/dist/ 目录不存在，需先构建前端：

```
cd frontend/
npm install
npm run build
cd ..
```

构建完成后 frontend/dist/ 目录将被后端自动检测并挂载为静态资源。

2.3.4 启动服务

```
cd deploy/  
docker compose --env-file .env up -d
```

首次启动流程：PostgreSQL 容器启动 → 健康检查通过 → Backend 构建镜像并启动 → lifespan 事件自动执行 create_all (建表) → seed_if_empty (种子数据，幂等)。

验证服务状态：

```
docker compose ps      # 查看所有容器状态，应有 db(healthy), redis(running),  
backend(running)  
curl http://localhost:8000/health # 返回 {"status":"ok","app":"...", "version":"0.1.0"}
```

2.4 开发环境（非 Docker）

开发模式使用 SQLite (默认) 代替 PostgreSQL，Vite 开发服务器代理 API 到后端。

```
终端 1 — 后端：  
cd backend/  
python -m venv venv && source venv/bin/activate  
pip install -r requirements.txt  
uvicorn app.main:app --reload --port 8000
```

```
终端 2 — 前端：  
cd frontend/  
npm install  
npm run dev  # Vite 开发服务器，默认 http://localhost:5173，自动代理 /api →  
localhost:8000
```

2.5 停止/重启/清理

操作	命令	说明
停止	docker compose down	停止并删除容器，保留数据卷
重启	docker compose restart	重启所有容器（不重建镜像）
重建并重启	docker compose up -d --build	代码更新后重建镜像并重启

彻底清理	<code>docker compose down -v</code>	停止容器并删除数据卷 (数据丢失 !)
查看日志	<code>docker compose logs -f backend</code>	实时跟踪后端日志
查看日志	<code>docker compose logs --tail=100 db</code>	查看 PostgreSQL 最近 100 行日志

第三章 环境配置

3.1 配置文件总览

文件路径	用途	优先级
deploy/.env	Docker 部署环境变量 (数据库密码、JWT 密钥)	最高
backend/app/core/config.py	Pydantic Settings 定义 (含默认值)	代码默认值
backend/alembic.ini	Alembic 数据库迁移配置	-
ml/params/evaluation_params.json	功能重构评价参数 (权重、分等赋值规则)	由 extract_params.py 生成
ml/params/reconstruction_scoring_rules.json	污染物分等赋值规则 (表 2.22 转录)	由文档转录固化

3.2 环境变量完整清单

变量名	默认值	作用域	必须	说明
POSTGRES_USER	srs	Docker	否	PostgreSQL 用户名
POSTGRES_PASSWORD	(无, 必须设置)	Docker	是	PostgreSQL 密码, docker-compose.yml 中约束为必填
POSTGRES_DB	srs	Docker	否	PostgreSQL 数据库名
SECRET_KEY	(无, 必须设置)	Docker	是	JWT 签名密钥, 至少 32 字符
DATABASE_URL	postgresql://...@db:5432/srs	Docker	否	SQLAlchemy 连接串 (Docker Compose 自动拼接)
REDIS_URL	redis://redis:6379/0	Docker	否	Redis 连接串
FILE_STORAGE_DIR	/app/storage	Docker	否	文件上传存储目录 (容器内

				路径)
APP_NAME	Soil Ecological- Production Function Reconstruction Regulation System	后端	否	FastAPI 应用名 称
API_V1_PREFIX	/api/v1	后端	否	API 版本前缀

第四章 巡检规程

4.1 巡检周期与项目

巡检项	周期	检查方法	正常指标	异常处理
容器状态	每日	docker compose ps	所有容器 Up/healthy	重启异常容器，见 5.1
磁盘使用率	每日	df -h /var/lib/docke r	< 80%	清理旧镜像/日志，见 5.4
后端健康检查	每日	curl http://localhost:8000/health	返回 {"status":"ok"}	检查后端日志，见 5.2
PostgreSQL 连接	每日	docker compose exec db pg_isready -U srs	accepting connections	检查 PG 日志，见 5.3
Redis 连接	每日	docker compose exec redis redis-cli PING	PONG	检查 Redis 日志
API 响应时间	每周	curl -w "@curl- format.txt" /api/v1/sites/	< 3s (MVP)	检查慢查询、网络
文件存储可用空间	每周	du -sh backend/storag e/	预留 2GB	清理过期临时文件
数据备份	每日	验证备份文件 存在且大小 > 0	见 6.2	重新执行备份
安全更新	每月	docker compose pull	有新镜像则构建测试	规划更新窗口
日志审查	每周	检查 audit_logs 异常操作	无可疑登录/批量导出	排查异常 IP

4.2 关键监控指标

指标类别	具体指标	告警阈值
API 可用性	/health 端点 HTTP 200	连续 3 次非 200
API 延迟	P95 响应时间	> 5s (MVP)
数据库连接	pg_isready 状态	连接拒绝
磁盘空间	Docker 数据卷使用率	> 85%
内存	容器内存使用	持续 > 80% 且未释放

文件存储	backend/storage/ 容量	< 1GB 可用
种子数据完整性	users 表至少有 admin 角色	查询返回 0 行

第五章 故障排查

5.1 容器无法启动

症状：docker compose up -d 后 backend 容器反复重启或处于 Exit 状态。

排查步骤：

1. 查看后端日志：docker compose logs backend --tail=50
2. 检查 .env 文件是否存在且 POSTGRES_PASSWORD、SECRET_KEY 是否正确设置
3. 检查 db 容器是否 healthy：docker compose ps db (健康检查需 pg_isready 通过)
4. 检查端口占用：lsof -i:8000, lsof -i:5432, lsof -i:6379
5. 常见报错：
 - "must set in .env" → .env 文件缺失或变量名拼写错误
 - "could not connect to server" → db 容器未 healthy, 等待 30s 后重启 backend
 - "relation already exists" → create_all 幂等, 可安全忽略

5.2 后端 API 返回 500

排查步骤：

1. docker compose logs backend --tail=100 查看堆栈
2. 常见原因：
 - 数据库连接断开 → backend 重启时会自动重连
 - 文件存储目录无写权限 → 检查 FILE_STORAGE_DIR 指向的目录权限
 - 模型文件缺失 → ml/artifacts/ 目录缺少 .joblib 文件, 需重新训练
 - 内存不足导致 PDF 生成失败 → WeasyPrint 渲染大报告需较多内存
3. 如需进入容器调试：

```
docker compose exec backend bash
python -c "from app.db.session import SessionLocal; s=SessionLocal();
s.execute(text('SELECT 1')); s.close()"
```

5.3 PostgreSQL 连接异常

排查步骤：

1. docker compose logs db --tail=50
2. docker compose exec db pg_isready -U srs

3. 常见报错：

- "too many connections" → 增加 max_connections 或排查连接泄漏
- "disk full" → pgdata 卷所在磁盘满，清理或扩容
- "could not write to WAL" → 磁盘空间不足或 I/O 异常

4. 如需重置数据库（会丢数据）：

```
docker compose down -v # 删除数据卷
docker compose up -d # 重建并重新初始化
```

5.4 磁盘空间不足

清理步骤（由安全到危险排序）：

- 1. 清理 Docker 构建缓存：docker builder prune -f
- 2. 清理未使用的镜像：docker image prune -f
- 3. 清理停止容器的卷：docker volume prune -f（谨慎：会删除未在使用的命名卷）
- 4. 检查 PostgreSQL 数据卷：docker system df -v | grep pgdata
- 5. 清理 PostgreSQL WAL 日志（容器内）：docker compose exec db bash -c "pg_archivecleanup /var/lib/postgresql/data/pg_wal \$(pg_controldata | grep 'Latest checkpoint' | cut -d: -f2)"

5.5 前端页面空白/资源 404

排查步骤：

- 1. 确认 frontend/dist/ 目录存在且含 index.html
- 2. 桌面打包模式下，main.py 启动日志会打印 FRONTEND_DIST 路径是否被检测到
- 3. 如果 frontend/dist/ 不存在，后端仅暴露 API，前端需单独通过 npm run dev 访问
- 4. 检查浏览器控制台（F12）Network 标签，确认是否有 /assets/ 或 /api/ 请求失败
- 5. CORS 问题：开发模式下 Vite dev server 在 5173 端口，后端 CORS 已白名单该端口

5.6 常见错误码

错误码	含义	处理建议
401 Unauthorized	JWT 令牌无效或过期	重新登录获取新令牌
403 Forbidden	无该操作权限	联系管理员分配权限
404 Not Found	资源不存在	检查 ID 是否正确或资源是

		否已删除
409 Conflict	资源冲突 (如重复 site_code)	使用唯一编号或先删除旧记录
422 Unprocessable Entity	请求参数校验失败	查看响应 body 中的字段级错误
500 Internal Server Error	服务器内部错误	查看后端日志，按 5.2 处理
503 Service Unavailable	依赖服务不可用 (如 DB 未就绪)	等待健康检查通过或排查 DB/Redis

第六章 备份恢复

6.1 备份策略

备份类型	周期	内容
数据库完整备份	每日	PostgreSQL pg_dump (全部数据)
文件存储备份	每日	backend/storage/ 目录 (上传的附件与报告)
配置文件备份	每次变更后	deploy/.env、backend/alembic.ini、ml/params/*.json
模型文件备份	每次训练后	ml/artifacts/*.joblib

6.2 备份操作命令

数据库备份 (在 deploy/ 目录执行) :

导出完整数据库

```
BACKUP_DIR="/path/to/backups"
mkdir -p "$BACKUP_DIR"
docker compose exec -T db pg_dump -U srs -d srs --clean --if-exists --format=custom \
> "$BACKUP_DIR/srs_db_$(date +%Y%m%d_%H%M%S).dump"
```

仅导出数据 (不含 DDL , 用于迁移)

```
docker compose exec -T db pg_dump -U srs -d srs --data-only --format=custom \
> "$BACKUP_DIR/srs_data_$(date +%Y%m%d_%H%M%S).dump"
```

导出为 SQL 文本 (可读 , 用于审查)

```
docker compose exec -T db pg_dump -U srs -d srs --clean --if-exists \
> "$BACKUP_DIR/srs_db_$(date +%Y%m%d_%H%M%S).sql"
```

文件存储备份 :

打包文件存储目录

```
tar -czf "$BACKUP_DIR/srs_storage_$(date +%Y%m%d_%H%M%S).tar.gz" \
/path/to/SRS/backend/storage/
```

建议设置 cron 自动执行 :

```
0 2 * * * /usr/local/bin/srs_backup.sh >> /var/log/srs_backup.log 2>&1
```

6.3 恢复操作

数据库恢复（在 deploy/ 目录执行）：

1. 停止后端（防止写入）

```
docker compose stop backend
```

2. 恢复数据库（从 custom 格式）

```
docker compose exec -T db pg_restore -U srs -d srs --clean --if-exists \  
< /path/to/backup/srs_db_YYYYMMDD_HHMMSS.dump
```

3. 如果是 SQL 文本格式

```
docker compose exec -T db psql -U srs -d srs \  
< /path/to/backup/srs_db_YYYYMMDD_HHMMSS.sql
```

4. 重启后端

```
docker compose start backend
```

5. 验证

```
curl http://localhost:8000/health
```

文件存储恢复：

```
tar -xzf /path/to/backup/srs_storage_YYYYMMDD_HHMMSS.tar.gz \  
-C /path/to/SRS/backend/
```

6.4 备份验证

每次备份后应验证：

检查备份文件大小 > 0

```
BACKUP_FILE="/path/to/backup/srs_db_20260613_020000.dump"  
test -s "$BACKUP_FILE" && echo "OK" || echo "FAIL: empty backup"
```

验证 pg_dump 文件完整性

```
docker compose exec -T db pg_restore --list "$BACKUP_FILE" > /dev/null 2>&1 && echo  
"OK" || echo "FAIL: corrupted"
```

验证文件存储备份

```
tar -tzf /path/to/backup/srs_storage_20260613_020000.tar.gz | head -5
```


第七章 应急预案

7.1 事故等级定义

等级	定义	影响范围	响应时限
P3 (严重)	数据库损坏或数据丢失、系统完全不可用超过 4 小时	全部用户	立即响应，2 小时内恢复
P2 (一般)	部分功能不可用 (如 PDF 报告生成失败、SHAP 分析超时) 但核心数据可访问	部分功能	1 小时内响应，4 小时内恢复
P1 (轻微)	非核心功能异常 (如地图显示异常、个别图表加载失败) 或性能下降	个别用户	下一个工作日处理

7.2 P3 严重事故处理流程

数据库损坏/数据丢失场景：

1. 立即通知用户系统暂停服务
2. 定位最近一次完整备份
3. 停止所有写操作容器：docker compose stop backend
4. 如 PostgreSQL 容器正常但数据损坏 → 执行 pg_restore (见 6.3)
5. 如 PostgreSQL 容器/数据卷也损坏：
docker compose down -v
编辑 docker-compose.yml，确认 pgdata 卷挂载路径
docker compose up -d db # 先启动新数据库
docker compose exec -T db pg_restore ... # 恢复数据
docker compose up -d # 启动全部服务
6. 验证：curl http://localhost:8000/health; 抽查核心数据
7. 通知用户恢复完成

7.3 P2 一般事故处理

PDF 报告生成失败：

1. 检查 WeasyPrint 系统依赖是否完整：docker compose exec backend python -c "import weasyprint; print('ok')"
2. 如报 libpango/libffi 缺失 → 重建镜像：docker compose build --no-cache backend
3. 检查数据完整性，确认生成所需的 measurements / diagnosis / evaluation 数据是否在库
4. 如系单场地数据异常，可先处理数据问题后重新生成

SHAP 分析超时：

1. 检查模型文件 ml/artifacts/ 是否存在
2. 检查采样点数量（大量采样点 → SHAP 计算耗时），考虑限制分析范围
3. 检查容器内存：docker stats backend

7.4 安全事件预案

安全事件	检测方法	应对措施
暴力登录	audit_logs 中频繁 login fail	临时锁定 IP / 加长密码最小长度
异常数据导出	audit_logs 中批量 export 操作	冻结涉事账号，审查导出记录
密钥泄露	.env 文件被不当访问	立即轮换 SECRET_KEY 和 POSTGRES_PASSWORD，作废所有现有 JWT
容器篡改	docker diff 检测文件变更	从官方镜像重建容器，排查入侵路径
依赖漏洞	docker scout / trivy 扫描	升级受影响的依赖包

第八章 附录

8.1 常用运维命令速查

命令	用途
<code>docker compose ps</code>	查看容器状态
<code>docker compose logs -f backend</code>	实时查看后端日志
<code>docker compose exec backend bash</code>	进入后端容器 Shell
<code>docker compose exec db psql -U srs -d srs</code>	进入 PostgreSQL 交互终端
<code>docker compose exec redis redis-cli</code>	进入 Redis 交互终端
<code>docker compose restart backend</code>	重启后端
<code>docker compose up -d --build</code>	重建镜像并启动
<code>docker compose down</code>	停止并删除容器 (保留数据卷)
<code>docker compose down -v</code>	停止并删除容器+数据卷 (数据丢失)
<code>docker system df</code>	查看 Docker 磁盘使用
<code>docker stats</code>	查看容器资源使用
<code>curl http://localhost:8000/health</code>	后端健康检查
<code>curl http://localhost:8000/api/v1/info</code>	系统信息
<code>docker compose exec db pg_dump -U srs -d srs > backup.sql</code>	导出数据库

8.2 关键文件路径速查

文件/目录	说明
<code>deploy/docker-compose.yml</code>	Docker 编排主文件
<code>deploy/.env</code>	环境变量 (密码、密钥, 不入 Git)
<code>backend/Dockerfile</code>	后端镜像构建
<code>backend/requirements.txt</code>	Python 依赖清单
<code>backend/app/main.py</code>	FastAPI 入口
<code>backend/app/core/config.py</code>	Pydantic Settings 配置定义
<code>backend/app/db/</code>	数据库会话、初始化、种子数据
<code>backend/alembic/</code>	Alembic 迁移脚本
<code>frontend/dist/</code>	前端构建产物 (桌面打包模式)
<code>data/raw/</code>	原始数据 (只读, 不可修改)
<code>data/knowledge_base/</code>	障碍因子知识库 V1.0
<code>ml/artifacts/</code>	训练好的模型文件 (*.joblib)

ml/params/	评价参数 (evaluation_params.json、scoring_rules.json)
ml/models/data_prep.py	数据预处理 (特征工程)
backend/storage/	文件存储 (容器挂载卷)

8.3 API 端点速查

路由前缀	模块	关键端点	
/api/v1/auth	认证	POST /login, POST /register, GET /me	
/api/v1/data	数据管理	GET/POST /sites, GET /sites/{id}, POST /import, GET /measurements	
/api/v1/diagnosis	障碍因子诊断	POST /run, GET /results/{id}, GET /factors/{diagnosis_id}	
/api/v1/evaluation	评价	POST /reconstruction, POST /ssui, GET /results/{id}	
/api/v1/workflow	全流程追溯	GET/POST /records, PUT /records/{id}, POST /attachments, GET /report/{site_id}	
/api/v1/system	系统管理	GET/POST /users, GET /audit-logs, GET/PUT /settings	
/api/v1/ai	AI 辅助	POST /chat, POST /summarize	
/health	健康检查	GET /health	
/api/v1/info	系统信息	GET /api/v1/info	

8.4 数据库核心表速查

表名	分组	用途
organizations	权限与组织	企业/机构信息
users	权限与组织	用户账户 (密码哈希存储)
roles / permissions	权限与组织	RBAC 角色与权限定义
sites	场地与检测	场地主表
sampling_points	场地与检测	采样点 (关联经纬度与采样

		深度)
factor_dictionary	场地与检测	因子字典 (122 因子 , 来自知识库 V1.0)
threshold_rules	场地与检测	阈值规则 (403 条 , pH 分段)
measurements	场地与检测	检测值长表 (核心 , 一污染物一行)
import_batches	场地与检测	导入批次溯源
ml_models	算法与评价	模型版本记录
diagnosis_results	算法与评价	RF+SHAP 诊断结果
diagnosis_factor_details	算法与评价	诊断因子明细 (含 SHAP 值)
evaluation_results	算法与评价	重构评价+SSUI 结果
technology_library	推荐	重构技术库
recommendations	推荐	方案推荐结果
workflow_records	追溯	五阶段追溯记录
workflow_attachments	追溯	阶段附件关联
report_records	追溯	报告生成记录
file_objects	系统	文件对象存储
audit_logs	系统	操作审计日志

8.5 联系信息

角色	姓名/组织	说明
项目负责人	裴总	甲方需求决策、功能确认
系统架构师/全栈开发	辛特助	系统架构、代码实现、算法设计
所属单位	浙江大学王玮课题组	开发与交付方

8.6 变更记录

版本	日期	变更	作者
v0.1	2026-06-13	初稿 , 含部署/配置/巡检/故障/备份/应急六章 + 附录	辛特助